

Informe Técnico: Análisis de la Red de Carreteras de Pennsylvania (roadNet-PA)

Estructuras de Datos y Algoritmos

Por: José David Acevedo, Nicolás Patiño Palacio

8/05/2026

1. Introducción

Este proyecto analiza una red vial real usando el dataset roadNet-PA del proyecto SNAP Stanford Network Analysis Project.

El grafo representa carreteras e intersecciones del estado de Pennsylvania, Estados Unidos. Cada nodo representa una intersección y cada arista representa una conexión vial entre dos puntos.

El objetivo principal fue aplicar algoritmos clásicos de grafos sobre un dataset de gran escala para entender cómo se comportan en escenarios reales y no solamente en ejemplos pequeños de clase.

También se utilizaron referencias del 9th DIMACS Implementation Challenge, uno de los benchmarks más importantes para evaluar algoritmos de rutas y caminos mínimos.

Este tipo de algoritmos tiene aplicaciones reales en:

- GPS y aplicaciones como Waze o Google Maps.
- Optimización de rutas de transporte.
- Redes de logística y distribución.
- Planeación urbana e infraestructura vial.

2. Descripción del Dataset

El dataset roadNet-PA contiene aproximadamente:

1,088,092 nodos

1,541,898 aristas

Al comparar los resultados obtenidos por nuestra implementación con los datos publicados por

SNAP, se obtuvo lo siguiente:

Propiedad	SNAP	Implementación
Nodos Totales	1,088,092	1,088,092
Aristas	1,541,898	1,541,898
Grado Máximo	9	9
Componentes Conexas	203	206
Diámetro Estimado	782	592

La diferencia de 3 componentes conexas (206 vs 203 publicado) se debe a pequeñas diferencias en el manejo de aristas duplicadas en la carga del archivo SNAP. El componente principal que son los nodos coincide como es esperado, lo cual indica de que la estructura central del grafo se cargó correctamente.

Propiedades de una Red Vial Real

Este grafo es una red dispersa, lo que significa que tiene muy pocas conexiones comparado con la cantidad máxima posible.

La mayoría de intersecciones solamente se conectan con pocas carreteras, normalmente entre 2 y 4 vecinos. Esto es muy diferente a las redes sociales, donde algunos nodos pueden tener miles o millones de conexiones.

Además, las redes viales suelen tener:

- Diámetros altos.
- Conectividad local.
- Restricciones geográficas.
- Caminos largos entre regiones lejanas.

Por esta razón, algoritmos eficientes son fundamentales e indispensables para procesar este tipo de información.

3. Diseño e Implementación

3.1 Representación del Grafo

Se utilizó una lista de adyacencia implementada con vectores dinámicos en C++.

Esta estructura fue elegida porque el grafo es disperso y permite ahorrar una enorme cantidad de memoria.

Una matriz de adyacencia requiere almacenar: $|V|^2$ entradas.

Para este dataset:

$(1,088,092)^2$

Esto requeriría aproximadamente:

Tipo de dato	Memoria Aproximada
bool	~1.18 TB
int	~4.72 TB

Esto hace que una matriz sea totalmente inviable en un computador normal.

En cambio, la lista de adyacencia utiliza:

$O(V+E)$

lo que reduce el consumo a aproximadamente 50–100 MB.

3.2. Seudocódigo de Algoritmos

Dijkstra con Heap Mínimo:

```
Función Dijkstra(Grafo G, Origen s):

distancia[V] = infinito
previo[V] = null
explorados = 0

distancia[s] = 0

PQ.push({0, s})
// Cola de prioridad mínima: (distancia, nodo)

Mientras PQ no esté vacía:

    d, u = PQ.pop_min()

    Si d > distancia[u]:
        continuar

    explorados++

    Para cada vecino v de u con peso w:

        Si distancia[u] + w < distancia[v]:

            distancia[v] = distancia[u] + w
            previo[v] = u

            PQ.push({distancia[v], v})
```

Estimación de Diámetro por BFS:

```
Función EstimarDiametro(Grafo G, NodoInicial s):
```

```
    Crear cola Q
```

```
    dist[V] = -1
```

```
    dist[s] = 0
```

```
    max_dist = 0
```

```
    Q.enqueue(s)
```

```
    Mientras Q no esté vacía:
```

```
        u = Q.dequeue()
```

```
        Para cada vecino v de u:
```

```
            Si dist[v] == -1:
```

```
                dist[v] = dist[u] + 1
```

```
                Q.enqueue(v)
```

```
                Si dist[v] > max_dist:
```

```
                    max_dist = dist[v]
```

```
    Retornar max_dist
```

4. Análisis de Complejidad

Complejidad de Dijkstra

Usando un heap (cola de prioridad), la complejidad temporal es:

$O((V+E) \log V)$

Esto ocurre porque:

- Cada nodo puede entrar al heap.
- Cada arista puede ser relajada una vez.
- Las operaciones del heap cuestan $\log(V)$.

Complejidad de BFS

La complejidad temporal de BFS es:

$$O(V+E)$$

Porque cada nodo y cada arista se visitan solamente una vez.

Estos algoritmos funcionan porque el grafo es disperso, entonces ambos algoritmos son viables incluso con más de un millón de nodos.

- Si se hubiera usado una matriz de adyacencia:
- el consumo de memoria sería enorme,
- las operaciones serían más lentas,
- y el programa probablemente no podría ejecutarse correctamente.

5. Resultados de Consultas P2P

1	consulta	origen	destino	dist_dijkstra	saltos_bfs	nodos_dijkstra	nodos_bfs	t_dijkstra_ms	t_bfs_ms
2	Q01	1	500000	529	101	74489	54713	14.1246	4.0223
3	Q02	100	1000000	2332	484	1121548	1008999	192.91	38.3175
4	Q03	50000	750000	2557	545	1029165	925233	168.02	34.3377
5	Q04	200000	800000	611	124	104461	88414	19.3229	4.2389
6	Q05	300000	100000	3098	666	1135889	1051816	195.291	41.7309
7	Q06	1	1087562	404	95	40205	47273	8.8999	3.0154
8	Q07	500000	1	529	101	111587	90936	20.9747	4.3946
9	Q08	250000	600000	1492	300	865235	758054	158.248	31.0446
10	Q09	10000	900000	889	169	201609	156513	34.7022	7.1066
11	Q10	400000	150000	805	167	390724	352068	72.3928	15.173

En este proyecto, el grafo fue tratado como no ponderado, por lo que todas las aristas tienen el mismo peso. Por esta razón:

Dijkstra y BFS producen las mismas distancias.

BFS resulta más rápido porque no necesita usar heap.

En promedio, BFS fue aproximadamente dos veces más rápido que Dijkstra.

Esto demuestra que:

- BFS es ideal cuando todos los pesos son iguales.
- Dijkstra es más general y útil cuando existen pesos diferentes.
- También se observó que las consultas más largas consumen más tiempo debido a la cantidad de nodos explorados.

6. Análisis del Subgrafo

Se construyó un subgrafo inducido usando las rutas encontradas en las consultas Q01 y Q06.

Árbol de Expansión Mínima (MST)

Usando el algoritmo de Kruskal se obtuvo un MST que conecta todos los nodos importantes utilizando la menor cantidad posible de aristas.

Geométricamente, esto representa una versión más sencilla de la red vial:

- mantiene conectividad,
- elimina redundancia,
- y minimiza conexiones innecesarias.

Este tipo de análisis es útil cuando:

- se hace diseño de infraestructura,
- instalación de fibra óptica,
- mantenimiento de las carreteras,
- y planeación de costos.

Detección de Ciclos

Usando DFS se detectó que el subgrafo contiene ciclos, por lo que no es un DAG.

Esto significa que existen múltiples rutas alternativas entre algunas intersecciones. Desde un punto de vista real, esto es importante porque: aumenta la robustez de la red, permite desvíos, y

evita que una sola carretera sea un punto crítico de falla.

7. Conclusiones

- Las listas de adyacencia son la mejor opción para trabajar con grafos reales de gran tamaño debido a su optimización de memoria.
- BFS fue más rápido que Dijkstra porque el grafo fue tratado como no ponderado, no toma en cuenta el peso de cada arista, lo cual para uso diario no es confiable.
- Los grafos reales son muy diferentes a los ejemplos pequeños vistos normalmente en clase, ya que tienen millones de nodos y requieren estructuras eficientes ya que la memoria y tiempo pueden escalar al infinito.
- Las redes viales son dispersas y tienen una estructura geográfica que afecta directamente el comportamiento de los algoritmos.
- El análisis de subgrafos, MST y ciclos permite entender mejor la organización y robustez de una red vial real.

8. Limitaciones del Proyecto

- El dataset no utilizó pesos reales como distancia física o tráfico.
- El diámetro obtenido mediante BFS es una aproximación.
- Los tiempos pueden variar dependiendo del computador usado.

9. Referencias

9.1 Dataset y benchmarks

[1] Leskovec, J. & Krevl, A. (2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. Stanford University. <http://snap.stanford.edu/data>

9.2 Algoritmos y estructuras de datos

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. Capítulos 22 (Búsqueda en grafos), 23 (Árboles de expansión mínima) y 24 (Caminos más cortos desde una sola fuente).

[3] Dijkstra, E. W. (1959). *A note on two problems in connection with graphs*. Numerische Mathematik, 1(1), 269–271.

[4] Kruskal, J. B. (1956). *On the shortest spanning subtree of a graph and the traveling salesman*

problem. Proceedings of the American Mathematical Society, 7(1), 48–50.

[5] Tarjan, R. E. (1975). *Efficiency of a good but not linear set union algorithm*. Journal of the ACM, 22(2), 215–225. (Path compression + union by rank en Disjoint Set Union).

[6] Álvarez Henao, C. (2024). *Notas de clase del curso Estructuras de Datos y Algoritmos 1*. Universidad EAFIT. Repositorio público en GitHub.

<https://github.com/carlosalvarezh/EstructuraDatosAlgoritmos1>

9.3 Documentación técnica de C++

[7] cppreference.com. *C++ Standard Library Reference*.

<https://en.cppreference.com/>

[8] GeeksforGeeks. *Priority Queue in C++ STL*.

<https://www.geeksforgeeks.org/cpp/priority-queue-in-cpp-stl/>

[9] GeeksforGeeks. *unordered_map in C++ STL*.

https://www.geeksforgeeks.org/cpp/unordered_map-in-cpp-stl/

[10] GeeksforGeeks. *File Handling through C++ Classes*.

<https://www.geeksforgeeks.org/cpp/file-handling-c-classes/>

9.4 Asistentes de IA

Como es solicitado en la sección 7 del documento, citamos el uso de herramientas de inteligencia artificial. Las decisiones de diseño, implementación y validación de los resultados fueron realizadas y verificadas por nosotros. A continuación, documentamos los usos puntuales que se les dio a estas herramientas:

[11] Anthropic. Claude (modelo Opus 4.7 Adaptive), 2026. Fue usado como tutor para aclarar y entender conceptos algorítmicos y revisar la lógica de los módulos. Ejemplos de prompts utilizados:

"Revisa nuestro README actual y sugiere cómo mejorarlo: qué información debe ir en el README y qué debe quedarse en el informe PDF."

"¿Cuál es la diferencia entre detectar ciclos en un grafo no dirigido (DFS normal) y verificar si

un grafo dirigido es DAG (DFS de tres colores)?"

"Revisa nuestra implementación de Kruskal y verifica que la deduplicación de aristas en el subgrafo esté correcta."

"Propón una estructura de pdf técnico de máximo 10 páginas para este proyecto, alineada con la rúbrica del proyecto."

[12] GitHub Copilot (integrado en Visual Studio Code), 2026. Microsoft & OpenAI. Fue usado para autocompletado de sintaxis (nombres de variables, plantillas básicas de bucles for y declaraciones de tipos STL). No se utilizó para generar lógica algorítmica del proyecto.

<https://github.com/features/copilot>